Definitions in `Sarason/` and `R2.lean`

Lean Formalization Project

November 3, 2026

This document provides a comprehensive list of all definitions manually formalized in the `ComplexAnalysis` project, specifically within `R2.lean` and the `Sarason/` directory. For each definition, the raw Lean code is presented first, followed by its mathematical interpretation.

1 `R2.lean`

- `sqNorm` ($x \in \mathbb{R}^2 \to \mathbb{R}$): The squared Euclidean norm.

```
noncomputable def sqNorm (x : ℝ × ℝ) : ℝ := x.1^2 + x.2^2
```

$$\text{sqNorm}(x) = x_1^2 + x_2^2$$

- `sqDist` ($x, y \in \mathbb{R}^2 \to \mathbb{R}$): The squared Euclidean distance.

```
noncomputable def sqDist (x y : ℝ × ℝ) : ℝ :=
  (x.1 - y.1)^2 + (x.2 - y.2)^2
```

$$\text{sqDist}(x, y) = \text{sqNorm}(x - y)$$

- `euclideanNorm` ($x \in \mathbb{R}^2 \to \mathbb{R}$): The standard Euclidean norm.

```
noncomputable def euclideanNorm (x : ℝ × ℝ) : ℝ :=
  Real.sqrt (sqNorm x)
```

$$\text{euclideanNorm}(x) = \sqrt{x_1^2 + x_2^2}$$

- `euclideanDist` ($x, y \in \mathbb{R}^2 \to \mathbb{R}$): The Euclidean distance.

```
noncomputable def euclideanDist (x y : ℝ × ℝ) : ℝ :=
  Real.sqrt (sqDist x y)
```

$$\text{euclideanDist}(x, y) = \sqrt{\text{sqDist}(x, y)}$$

- `LimitRtoR2` ($f : \mathbb{R} \to \mathbb{R}^2$, $a \in \mathbb{R}$, $L \in \mathbb{R}^2$):

```
def LimitRtoR2 (f : ℝ → ℝ × ℝ) (a : ℝ) (L : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - a| ∧ |x - a| < δ → euclideanDist (f x) L
    < ε
```

$$\lim_{x \to a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x, 0 < |x - a| < \delta \implies \text{euclideanDist}(f(x), L) < \epsilon$$

- `LimitR2toR` ($f : \mathbb{R}^2 \to \mathbb{R}$, $a \in \mathbb{R}^2$, $L \in \mathbb{R}$):

```
def LimitR2toR (f : ℝ × ℝ → ℝ) (a : ℝ × ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ × ℝ, 0 < euclideanDist x a ∧ euclideanDist x a < δ
    → |f x - L| < ε
```

$$\lim_{x\to a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x \in \mathbb{R}^2, 0 < \text{euclideanDist}(x, a) < \delta \implies |f(x) - L| < \epsilon$$

- `LimitR2toR2` ($f : \mathbb{R}^2 \to \mathbb{R}^2$, $a \in \mathbb{R}^2$, $L \in \mathbb{R}^2$):

```
def LimitR2toR2 (f : ℝ × ℝ → ℝ × ℝ) (a : ℝ × ℝ) (L : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ × ℝ, 0 < euclideanDist x a ∧ euclideanDist x a < δ
    → euclideanDist (f x) L < ε
```

$$\lim_{x\to a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x \in \mathbb{R}^2, 0 < \text{euclideanDist}(x, a) < \delta \implies \text{euclideanDist}(f(x), L) < \epsilon$$

- `ConvergesR2` ($\text{seq} : \mathbb{N} \to \mathbb{R}^2$, $L \in \mathbb{R}^2$):

```
def ConvergesR2 (seq : ℕ → ℝ × ℝ) (L : ℝ × ℝ): Prop :=
  ∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, euclideanDist (seq n) L < ε
```

$$\lim_{n\to\infty} \text{seq}(n) = L \iff \forall \epsilon > 0, \exists N \in \mathbb{N}, \forall n \geq N, \text{euclideanDist}(\text{seq}(n), L) < \epsilon$$

- `HasFDerivAt_R2_eps` ($u, v : \mathbb{R}^2 \to \mathbb{R}$, $u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}$, $a \in \mathbb{R}^2$): Defines the Fréchet derivative of $F(x, y) = (u(x, y), v(x, y))$ at a. Let $L(h) = \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} h$.

```
def HasFDerivAt_R2_eps (u v : ℝ × ℝ → ℝ) (ux₀ uy₀ vx₀ vy₀ : ℝ) (a : ℝ × ℝ) :
    Prop :=
  LimitR2toR (fun h =>
    euclideanNorm (
      (u (a.1 + h.1, a.2 + h.2) - u a) - (ux₀ * h.1 + uy₀ * h.2),
      (v (a.1 + h.1, a.2 + h.2) - v a) - (vx₀ * h.1 + vy₀ * h.2)
    ) / euclideanNorm h
  ) (0, 0) 0
```

$$\lim_{h\to 0} \frac{\left\| F(a+h) - F(a) - \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} h \right\|}{\|h\|} = 0$$

$$\lim_{(h_1,h_2)\to(0,0)} \frac{\left\| \begin{pmatrix} u(a_1+h_1, a_2+h_2) - u(a_1, a_2) \\ v(a_1+h_1, a_2+h_2) - v(a_1, a_2) \end{pmatrix} - \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \end{pmatrix} \right\|}{\|(h_1, h_2)\|} = 0$$

- `DifferentiableAt_R2_eps` ($u, v : \mathbb{R}^2 \to \mathbb{R}$, $a \in \mathbb{R}^2$):

```
def DifferentiableAt_R2_eps (u v : ℝ × ℝ → ℝ) (a : ℝ × ℝ) : Prop :=
  ∃ (ux₀ uy₀ vx₀ vy₀ : ℝ), HasFDerivAt_R2_eps u v ux₀ uy₀ vx₀ vy₀ a
```

$$\exists u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}, \text{HasFDerivAt_R2_eps}(u, v, u_{x0}, u_{y0}, v_{x0}, v_{y0}, a)$$

- `HasDerivAt_RtoR2_eps` ($f : \mathbb{R} \to \mathbb{R}^2$, $f_0' \in \mathbb{R}^2$, $t_0 \in \mathbb{R}$): Defines the derivative of a parametric curve in $\mathbb{R}^2$.

```
def HasDerivAt_RtoR2_eps (f : ℝ → ℝ × ℝ) (f'₀ : ℝ × ℝ) (t₀ : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ t : ℝ, 0 < |t - t₀| ∧ |t - t₀| < δ →
    euclideanDist (f t) (f t₀ + (f'₀.1 * (t - t₀), f'₀.2 * (t - t₀))) < ε * |t
    - t₀|
```

$$\lim_{t \to t_0} (f(t) - f(t_0) - (t - t_0) f'_0) = 0$$

- `HasPartialDerivX_R2_eps` ($u : \mathbb{R}^2 \to \mathbb{R}$, $ux_0 \in \mathbb{R}$, $p_0 \in \mathbb{R}^2$):

```
def HasPartialDerivX_R2_eps (u : ℝ × ℝ → ℝ) (ux₀ : ℝ) (p₀ : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - p₀.1| ∧ |x - p₀.1| < δ →
    |(u (x, p₀.2) - u p₀) / (x - p₀.1) - ux₀| < ε
```

$$\lim_{x \to p_{0,x}} \frac{u(x, p_{0,y}) - u(p_0)}{x - p_{0,x}} = ux_0$$

- `HasPartialDerivY_R2_eps` ($u : \mathbb{R}^2 \to \mathbb{R}$, $uy_0 \in \mathbb{R}$, $p_0 \in \mathbb{R}^2$):

```
def HasPartialDerivY_R2_eps (u : ℝ × ℝ → ℝ) (uy₀ : ℝ) (p₀ : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ y : ℝ, 0 < |y - p₀.2| ∧ |y - p₀.2| < δ →
    |(u (p₀.1, y) - u p₀) / (y - p₀.2) - uy₀| < ε

/-- A real-valued function of two real variables is harmonic on an open set G
if its first and second partial derivatives are continuous and satisfy Laplace
   's equation. -/
```

$$\lim_{y \to p_{0,y}} \frac{u(p_{0,x}, y) - u(p_0)}{y - p_{0,y}} = uy_0$$

- `TwiceContinuouslyDifferentiable_R2_eps` ($u : \mathbb{R}^2 \to \mathbb{R}$, $G \subset \mathbb{R}^2$): Defines twice continuous differentiability (C^2) for a real-valued function on an open subset $G \subset \mathbb{R}^2$.

```
def TwiceContinuouslyDifferentiable_R2_eps (u : ℝ × ℝ → ℝ) (G : Set (ℝ × ℝ))
    : Prop :=
  IsOpen G ∧
  ∃ (ux uy uxx uxy uyx uyy : ℝ × ℝ → ℝ),
    (∀ p ∈ G, HasPartialDerivX_R2_eps u (ux p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps u (uy p) p) ∧
    (ContinuousOn ux G) ∧
    (ContinuousOn uy G) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps ux (uxx p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps ux (uxy p) p) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps uy (uyx p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps uy (uyy p) p) ∧
    (ContinuousOn uxx G) ∧
    (ContinuousOn uxy G) ∧
    (ContinuousOn uyx G) ∧
    (ContinuousOn uyy G)
```

$u \in C^2(G, \mathbb{R}) \iff u_x, u_y, u_{xx}, u_{xy}, u_{yx}, u_{yy}$ exist and are continuous on the open set G

- `Harmonic_R2_eps` ($u : \mathbb{R}^2 \to \mathbb{R}$, $G \subset \mathbb{R}^2$):

```
def Harmonic_R2_eps (u : ℝ × ℝ → ℝ) (G : Set (ℝ × ℝ)) : Prop :=
  IsOpen G ∧
  ∃ (ux uy uxx uxy uyx uyy : ℝ × ℝ → ℝ),
    (∀ p ∈ G, HasPartialDerivX_R2_eps u (ux p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps u (uy p) p) ∧
    (ContinuousOn ux G) ∧
    (ContinuousOn uy G) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps ux (uxx p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps ux (uxy p) p) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps uy (uyx p) p) ∧
```

```
    (∀ p ∈ G, HasPartialDerivY_R2_eps uy (uyy p) p) ∧
    (ContinuousOn uxx G) ∧
    (ContinuousOn uxy G) ∧
    (ContinuousOn uyx G) ∧
    (ContinuousOn uyy G) ∧
    (∀ p ∈ G, uxx p + uyy p = 0)
```

First and second partials continuous on G, $\quad \Delta u = u_{xx} + u_{yy} = 0$

2 Sarason/Definitions.lean

- `HasLimitAt_eps` ($f : \mathbb{C} \to \mathbb{C}$, $L \in \mathbb{C}$, $z_0 \in \mathbb{C}$):

```
def HasLimitAt_eps (f : ℂ → ℂ) (L : ℂ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ z, 0 < ‖z - z₀‖ ∧ ‖z - z₀‖ < δ → ‖f z - L‖ < ε

--  Weierstrass definition of a limit at infinity.
```

$$\lim_{z \to z_0} f(z) = L$$

- `HasLimitAtInf_eps` ($f : \mathbb{C} \to \mathbb{R}$, $L \in \mathbb{R}$):

```
def HasLimitAtInf_eps (f : ℂ → ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ M > 0, ∀ z, ‖z‖ > M → |f z - L| < ε

--  Weierstrass definition of having a complex derivative at z₀.
```

$$\lim_{|z| \to \infty} f(z) = L$$

- `HasDerivAt_eps` ($f : \mathbb{C} \to \mathbb{C}$, $f_0' \in \mathbb{C}$, $z_0 \in \mathbb{C}$): Complex differentiability at a point.

```
def HasDerivAt_eps (f : ℂ → ℂ) (f'₀ : ℂ) (z₀ : ℂ) : Prop :=
  HasLimitAt_eps (fun z => (f z - f z₀) / (z - z₀)) f'₀ z₀

--  Weierstrass definition of complex differentiability at z₀.
```

$$\lim_{z \to z_0} \frac{f(z) - f(z_0)}{z - z_0} = f_0'$$

- `DifferentiableAt_eps` ($f : \mathbb{C} \to \mathbb{C}$, $z_0 \in \mathbb{C}$):

```
def DifferentiableAt_eps (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  ∃ f'₀, HasDerivAt_eps f f'₀ z₀

-- Weierstrass definition of partial derivative with respect to x.
```

f is differentiable at z_0

- `HasPartialDerivX_C_to_R_eps` / `HasPartialDerivY_C_to_R_eps`: Partial derivatives of a real-valued function of a complex variable (identifying $\mathbb{C} \cong \mathbb{R}^2$).

```
def HasPartialDerivX_C_to_R_eps (u : ℂ → ℝ) (ux₀ : ℝ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - z₀.re| ∧ |x - z₀.re| < δ →
    |(u ((x : ℂ) + (z₀.im : ℂ) * I) - u z₀) / (x - z₀.re) - ux₀| < ε

-- Weierstrass definition of partial derivative with respect to y.
```

$$\frac{\partial u}{\partial x}(z_0) = u_{x0}$$

- `HasPartialDerivY_C_to_R_eps`

```
def HasPartialDerivY_C_to_R_eps (u : ℂ → ℝ) (uy₀ : ℝ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ y : ℝ, 0 < |y - z₀.im| ∧ |y - z₀.im| < δ →
    |(u ((z₀.re : ℂ) + (y : ℂ) * I) - u z₀) / (y - z₀.im) - uy₀| < ε

--  Equivalence theorem between Mathlib's limit at a point (using punctured
    neighborhoods) and Weierstrass epsilon-delta limit.
```

$$\frac{\partial u}{\partial y}(z_0) = u_{y0}$$

- `deriv_eps` ($f : \mathbb{C} \to \mathbb{C}$, $z_0 \in \mathbb{C}$, h : `DifferentiableAt_eps` f z_0): The classical derivative value $f'(z_0)$ extracted via choice.

```
noncomputable def deriv_eps (f : ℂ → ℂ) (z₀ : ℂ) (h : DifferentiableAt_eps f
    z₀) : ℂ :=
  Classical.choose h
```

$$\text{deriv_eps}(f, z_0) = f'(z_0)$$

- `HasDerivAt_R_to_C_eps` ($f : \mathbb{R} \to \mathbb{C}$, $f'_0 \in \mathbb{C}$, $t_0 \in \mathbb{R}$): Derivative of a complex-valued path parameterized by a real variable.

```
def HasDerivAt_R_to_C_eps (f : ℝ → ℂ) (f'₀ : ℂ) (t₀ : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ t : ℝ, 0 < |t - t₀| ∧ |t - t₀| < δ → ‖(f t - f t₀) / ((t
    - t₀ : ℝ) : ℂ) - f'₀‖ < ε
```

$$\lim_{t \to t_0} \frac{f(t) - f(t_0)}{t - t_0} = f'_0$$

- `DifferentiableAt_R_to_C_eps` ($f : \mathbb{R} \to \mathbb{C}$, $t_0 \in \mathbb{R}$):

```
def DifferentiableAt_R_to_C_eps (f : ℝ → ℂ) (t₀ : ℝ) : Prop :=
  ∃ f'₀, HasDerivAt_R_to_C_eps f f'₀ t₀

/-- The classical derivative value extracted via choice, or 0 if not
    differentiable. -/
```

$$\exists f'_0 \in \mathbb{C}, \text{HasDerivAt_R_to_C_eps}(f, f'_0, t_0)$$

- `deriv_R_to_C_eps` ($f : \mathbb{R} \to \mathbb{C}$, $t_0 \in \mathbb{R}$):

```
noncomputable def deriv_R_to_C_eps (f : ℝ → ℂ) (t₀ : ℝ) : ℂ :=
  if h : DifferentiableAt_R_to_C_eps f t₀ then Classical.choose h else 0
```

$$\text{deriv_R_to_C_eps}(f, t_0) = \begin{cases} f'(t_0) & \text{if differentiable} \\ 0 & \text{otherwise} \end{cases}$$

- `HolomorphicAt_eps` ($f : \mathbb{C} \to \mathbb{C}$, $z_0 \in \mathbb{C}$): f is complex-differentiable in some open neighborhood around z_0.

```
def HolomorphicAt_eps (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  DifferentiableAt_eps f z₀

/-- A function is holomorphic on a set if it is complex differentiable at
    every point of the set. -/
```

f is holomorphic at $z_0 \iff f$ is complex-differentiable on some open ball around z_0

- `HolomorphicOn_eps` ($f : \mathbb{C} \to \mathbb{C}$, $G \subset \mathbb{C}$): f is holomorphic at every point in the open set G.

```
def HolomorphicOn_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ∀ z ∈ G, DifferentiableAt_eps f z
```

f is holomorphic on G

- `polar_partial_r` ($ux, uy : \mathbb{R}^2 \to \mathbb{R}$, $p \in \mathbb{R}^2$): Directional derivative in the radial direction for polar coordinates.

```
noncomputable def polar_partial_r (ux uy : ℝ × ℝ → ℝ) (p : ℝ × ℝ) : ℝ :=
  ux (p.1 * Real.cos p.2, p.1 * Real.sin p.2) * Real.cos p.2 +
  uy (p.1 * Real.cos p.2, p.1 * Real.sin p.2) * Real.sin p.2
```

$$U_r(r, \theta) = u_x(r\cos\theta, r\sin\theta)\cos\theta + u_y(r\cos\theta, r\sin\theta)\sin\theta$$

- `polar_partial_theta` ($ux, uy : \mathbb{R}^2 \to \mathbb{R}$, $p \in \mathbb{R}^2$): Directional derivative in the angular direction for polar coordinates.

```
noncomputable def polar_partial_theta (ux uy : ℝ × ℝ → ℝ) (p : ℝ × ℝ) : ℝ
    :=
  -ux (p.1 * Real.cos p.2, p.1 * Real.sin p.2) * p.1 * Real.sin p.2 +
  uy (p.1 * Real.cos p.2, p.1 * Real.sin p.2) * p.1 * Real.cos p.2
```

$$U_\theta(r, \theta) = -r\sin\theta\ u_x(r\cos\theta, r\sin\theta) + r\cos\theta\ u_y(r\cos\theta, r\sin\theta)$$

- `polar_partial_rr` ($uxx, uxy, uyy : \mathbb{R}^2 \to \mathbb{R}$, $p \in \mathbb{R}^2$): Second partial derivative with respect to r in polar coordinates.

```
noncomputable def polar_partial_rr (uxx uxy uyy : ℝ × ℝ → ℝ) (p : ℝ × ℝ) :
    ℝ :=
  Real.cos p.2 * polar_partial_r uxx uxy p + Real.sin p.2 * polar_partial_r
    uxy uyy p
```

$$U_{rr}(r, \theta) = \cos\theta\,(u_{xx})_r + \sin\theta\,(u_{xy})_r = \cos^2\theta\,u_{xx} + 2\sin\theta\cos\theta\,u_{xy} + \sin^2\theta\,u_{yy}$$

- `polar_partial_thetatheta` ($ux, uy, uxx, uxy, uyy : \mathbb{R}^2 \to \mathbb{R}$, $p \in \mathbb{R}^2$): Second partial derivative with respect to θ in polar coordinates.

```
noncomputable def polar_partial_thetatheta (ux uy uxx uxy uyy : ℝ × ℝ → ℝ) (
    p : ℝ × ℝ) : ℝ :=
  -p.1 * Real.cos p.2 * ux (p.1 * Real.cos p.2, p.1 * Real.sin p.2)
  - p.1 * Real.sin p.2 * polar_partial_theta uxx uxy p
  - p.1 * Real.sin p.2 * uy (p.1 * Real.cos p.2, p.1 * Real.sin p.2)
  + p.1 * Real.cos p.2 * polar_partial_theta uxy uyy p
```

$$U_{\theta\theta}(r, \theta) = -r(u_x\cos\theta + u_y\sin\theta) - r\sin\theta\,(u_{xx})_\theta + r\cos\theta\,(u_{xy})_\theta$$

- `TwiceContinuouslyDifferentiable_C_eps` ($f : \mathbb{C} \to \mathbb{C}$, $G \subset \mathbb{C}$): f is twice continuously differentiable on the open set G if its real and imaginary parts are.

```
def TwiceContinuouslyDifferentiable_C_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ComplexAnalysis.R2.TwiceContinuouslyDifferentiable_R2_eps (fun p => (f (p.1
    + p.2 * I)).re) {p : ℝ × ℝ | p.1 + p.2 * I ∈ G} ∧
  ComplexAnalysis.R2.TwiceContinuouslyDifferentiable_R2_eps (fun p => (f (p.1
    + p.2 * I)).im) {p : ℝ × ℝ | p.1 + p.2 * I ∈ G}
```

$$\mathrm{Re}(f), \mathrm{Im}(f) \in C^2(G, \mathbb{R})$$

- `Harmonic_C_eps` ($f : \mathbb{C} \to \mathbb{C}$, $G \subset \mathbb{C}$): f is harmonic on the open set G if its real and imaginary parts satisfy Laplace's equation $\Delta = 0$ on G.

```
def Harmonic_C_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => (f (p.1 + p.2 * I)).re) {p : ℝ
    × ℝ | p.1 + p.2 * I ∈ G} ∧
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => (f (p.1 + p.2 * I)).im) {p : ℝ
    × ℝ | p.1 + p.2 * I ∈ G}
```

$$\Delta \mathrm{Re}(f) = 0, \quad \Delta \mathrm{Im}(f) = 0 \text{ on } G$$

- `HarmonicConjugate_C_eps` ($u, v : \mathbb{C} \to \mathbb{R}$, $G \subset \mathbb{C}$): A real-valued function v is a harmonic conjugate of a real-valued function u on an open set G if both are harmonic on G and they satisfy the Cauchy-Riemann equations on G.

```
def HarmonicConjugate_C_eps (u v : ℂ → ℝ) (G : Set ℂ) : Prop :=
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => u (p.1 + p.2 * I)) {p : ℝ × ℝ
    | p.1 + p.2 * I ∈ G} ∧
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => v (p.1 + p.2 * I)) {p : ℝ × ℝ
    | p.1 + p.2 * I ∈ G} ∧
  ∃ (ux uy vx vy : ℂ → ℝ),
    (∀ z ∈ G, HasPartialDerivX_C_to_R_eps u (ux z) z) ∧
    (∀ z ∈ G, HasPartialDerivY_C_to_R_eps u (uy z) z) ∧
    (∀ z ∈ G, HasPartialDerivX_C_to_R_eps v (vx z) z) ∧
    (∀ z ∈ G, HasPartialDerivY_C_to_R_eps v (vy z) z) ∧
    (∀ z ∈ G, ux z = vy z ∧ uy z = -vx z)
```

$$\Delta u = 0, \quad \Delta v = 0, \quad u_x = v_y, \quad u_y = -v_x \text{ on } G$$

3 Sarason/Chapter1.lean

- `chordalMetric` ($z, w \in \mathbb{C} \to \mathbb{R}$): The chordal distance between two points in the complex plane mapped to the Riemann sphere.

```
noncomputable def chordalMetric (z w : ℂ) : ℝ :=
  (2 * Complex.abs (z - w)) / (Real.sqrt (1 + Complex.normSq z) * Real.sqrt (1
    + Complex.normSq w))
```

$$\chi(z, w) = \frac{2|z - w|}{\sqrt{1 + |z|^2}\sqrt{1 + |w|^2}}$$

- `chordalMetricToInf` ($z \in \mathbb{C} \to \mathbb{R}$): The chordal distance from a point z to the point at infinity ∞.

```
noncomputable def chordalMetricToInf (z : ℂ) : ℝ :=
  2 / Real.sqrt (1 + Complex.normSq z)
```

$$\chi(z, \infty) = \frac{2}{\sqrt{1 + |z|^2}}$$

- `Φ (Stereographic Projection)`

```
noncomputable def Φ (z : ℂ) : EuclideanSpace ℝ (Fin 3) :=
  ![ (2 * z.re) / (1 + Complex.normSq z),
     (2 * z.im) / (1 + Complex.normSq z),
     (Complex.normSq z - 1) / (1 + Complex.normSq z) ]
```

$$\Phi(z) = \left(\frac{2\,\mathrm{Re}(z)}{1+|z|^2}, \frac{2\,\mathrm{Im}(z)}{1+|z|^2}, \frac{|z|^2-1}{1+|z|^2} \right) \in S^2$$

- `ChordalC`: Type alias for $\mathbb{C}$ equipped with the chordal metric.

```
def Chordalℂ : Type := ℂ
```

$$\text{ChordalC} = \mathbb{C} \text{ (equipped with chordal metric)}$$

4 Sarason/Chapter2.lean

- `del` (∂)

```
def del (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  let L := fderiv ℝ f z
  let df_dx := L 1
  let df_dy := L I
  (1 / 2 : ℂ) * (df_dx - I * df_dy)

/-- The ∂ operator (del-bar) for a function f : ℂ → ℂ at point z. -/
```

$$\partial f = \frac{1}{2}\left(\frac{\partial f}{\partial x} - i \frac{\partial f}{\partial y} \right)$$

- `delBar` ($\bar{\partial}$)

```
def delBar (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  let L := fderiv ℝ f z
  let df_dx := L 1
  let df_dy := L I
  (1 / 2 : ℂ) * (df_dx + I * df_dy)

-- Local notation for Sarason's flavor
local notation f "z" => del f
local notation f "zbar" => delBar f

/--
  Relating Sarason's Weierstrass derivative to Mathlib's Filter-based
    HasDerivAt.
-/
```

$$\bar{\partial} f = \frac{1}{2}\left(\frac{\partial f}{\partial x} + i \frac{\partial f}{\partial y} \right)$$

- `ConformalLinearMap_eps` ($ux, uy, vx, vy \in \mathbb{R}$, $v \in \mathbb{C}$): Real-linear transformation on $\mathbb{C} \cong \mathbb{R}^2$ specified by the 2×2 Jacobian entries (u_x, u_y, v_x, v_y).

```
def ConformalLinearMap_eps (ux uy vx vy : ℝ) (v : ℂ) : ℂ :=
  (ux * v.re + uy * v.im) + I * (vx * v.re + vy * v.im)
```

$$L(v) = (u_x\,\mathrm{Re}(v) + u_y\,\mathrm{Im}(v)) + i(v_x\,\mathrm{Re}(v) + v_y\,\mathrm{Im}(v)) = (\partial f)\,v + (\bar{\partial} f)\,\bar{v}$$

- `del_eps` / `delBar_eps` ($u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}$): Wirtinger derivatives expressed purely in terms of the real partial derivatives of $f = u + iv$.

```
noncomputable def del_eps (ux uy vx vy : ℝ) : ℂ :=
  (1 / 2 : ℂ) * ( (ux + vy) + I * (vx - uy) )
```

$$\partial f = \frac{1}{2}(u_{x0} + v_{y0}) + \frac{i}{2}(v_{x0} - u_{y0})$$

- `delBar_eps`

```
noncomputable def delBar_eps (ux uy vx vy : ℝ) : ℂ :=
  (1 / 2 : ℂ) * ( (ux - vy) + I * (vx + uy) )
```

$$\bar{\partial} f = \frac{1}{2}(u_{x0} - v_{y0}) + \frac{i}{2}(v_{x0} + u_{y0})$$

- `path_in_C` ($x, y \in \mathbb{C}$): A continuous path $\gamma : [0,1] \to \mathbb{C}$ starting at x and ending at y.

```
def path_in_C (x y : ℂ) : Type := Path x y

/-- Velocity / derivative of a path γ : path_in_C x y at parameter time t ∈ ℝ
    . -/
```

$$\gamma : [0,1] \to \mathbb{C} \text{ where } \gamma(0) = x, \gamma(1) = y$$

- `translated_path` ($c, x, y \in \mathbb{C}$, γ : path_in_C x y): A path shifted by the constant c, creating a path from $c + x$ to $c + y$.

```
noncomputable def translated_path {x y : ℂ} (c : ℂ) (γ : path_in_C x y) :
    path_in_C (c + x) (c + y) where
  toFun := fun t => c + γ.toFun t
  continuous_toFun := Continuous.add continuous_const γ.continuous_toFun
  source' := by { change c + γ.toFun 0 = c + x; have h : γ.toFun 0 = x := (γ :
     Path x y).source'; rw [h] }
  target' := by { change c + γ.toFun 1 = c + y; have h : γ.toFun 1 = y := (γ :
     Path x y).target'; rw [h] }
```

$$\gamma_c(t) = c + \gamma(t)$$

- `pathDeriv` (γ : path_in_C x y, $t \in \mathbb{R}$): The tangent vector $\gamma'(t) \in \mathbb{C}$, extracted via choice from `HasDerivAt_R_to_C_eps` extended to $\mathbb{R}$.

```
noncomputable def pathDeriv {x y : ℂ} (γ : path_in_C x y) (t : ℝ) : ℂ :=
  deriv_R_to_C_eps γ.extend t

/-- The direction of a path γ at time t is the argument of its derivative. -/
```

$$\gamma'(t) = \frac{d\gamma}{dt}$$

- `pathDirection` (γ : path_in_C x y, $t \in \mathbb{R}$): The argument (angle) of the tangent vector, $\arg(\gamma'(t))$.

```
noncomputable def pathDirection {x y : ℂ} (γ : path_in_C x y) (t : ℝ) : ℝ :=
  arg (pathDeriv γ t)

/-- The angle between two paths γ₁ and γ₂ at times t₁ and t₂ is the difference
    of their directions. -/
```

$$\text{Direction}(\gamma, t) = \arg(\gamma'(t))$$

- `pathAngle` ($\gamma_1, t_1, \gamma_2, t_2$): The angle between two intersecting paths.

```
noncomputable def pathAngle {x₁ y₁ x₂ y₂ : ℂ} (γ₁ : path_in_C x₁ y₁) (t₁ : ℝ) (
    γ₂ : path_in_C x₂ y₂) (t₂ : ℝ) : ℝ :=
  pathDirection γ₂ t₂ - pathDirection γ₁ t₁

/--
  The angle between two paths is equal to the argument of the product of the
    second path's derivative
  and the conjugate of the first path's derivative, modulo 2π.
  We prove this equality here assuming the angle difference stays within the
    principal branch bounds.
-/
```

$$\text{Angle}(\gamma_1, t_1, \gamma_2, t_2) = \arg(\gamma_2'(t_2)) - \arg(\gamma_1'(t_1))$$

- `conformal` ($f : \mathbb{C} \to \mathbb{C}$, $z_0 \in \mathbb{C}$): f preserves angles between any two paths intersecting at z_0 with non-zero derivatives.

```
def conformal (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  ∀ (x₁ y₁ x₂ y₂ : ℂ) (γ₁ : path_in_C x₁ y₁) (t₁ : ℝ) (γ₂ : path_in_C x₂ y₂) (t₂
    : ℝ) (d₁ d₂ : ℂ),
    γ₁.extend t₁ = z₀ →
    γ₂.extend t₂ = z₀ →
    HasDerivAt_R_to_C_eps γ₁.extend (pathDeriv γ₁ t₁) t₁ →
    HasDerivAt_R_to_C_eps γ₂.extend (pathDeriv γ₂ t₂) t₂ →
    pathDeriv γ₁ t₁ ≠ 0 →
    pathDeriv γ₂ t₂ ≠ 0 →
    -- HasDerivAt_R_to_C_eps f f' t says f' is the derivative of f at t.
    HasDerivAt_R_to_C_eps (f ∘ γ₁.extend) d₁ t₁ →
    HasDerivAt_R_to_C_eps (f ∘ γ₂.extend) d₂ t₂ →
    d₁ ≠ 0 ∧ d₂ ≠ 0 ∧ arg d₂ - arg d₁ = pathAngle γ₁ t₁ γ₂ t₂

/--
  \SII.11 Angle preservation at a point.  CONFORMALITY.
  If f is holomorphic at z₀ with non-zero derivative, then it preserves angles
     between regular paths intersecting at z₀.
-/
```

Conformal f at $z_0 \iff f$ preserves angles between all regular intersecting curves at z_0

- `line_path_zero` ($v \in \mathbb{C}$): A straight-line path from $-v$ to v passing through 0 at $t = 1/2$.

```
noncomputable def line_path_zero (v : ℂ) : path_in_C (-v) v where
  toFun := fun t => -v + (2 * ((t : ℝ) : ℂ)) * v
```

$$\gamma(t) = -v + 2tv = (2t - 1)v$$

- `centered_line_path` ($z, v \in \mathbb{C}$): A straight-line path from $z - v$ to $z + v$ evaluating to z at $t = 1/2$.

```
noncomputable def centered_line_path (z v : ℂ) : path_in_C (z - v) (z + v)
    where
  toFun := fun t => z + (2 * (t : ℝ) - 1) * v
```

$$\gamma(t) = z + (2t - 1)v$$

5 Sarason/Chapter3.lean

- `linFracTrans` ($a, b, c, d, z \in \mathbb{C}$): A linear-fractional transformation (Möbius transformation) on $\mathbb{C}$.

```
noncomputable def linFracTrans (a b c d z : ℂ) : ℂ :=
  (a * z + b) / (c * z + d)
```

$$\varphi(z) = \frac{az + b}{cz + d}$$

- φ ($a, b, c, d, z \in \mathbb{C}$): Sarason's shorthand notation for `linFracTrans`.

```
noncomputable abbrev φ (a b c d z : ℂ) : ℂ :=
  linFracTrans a b c d z
```

$$\varphi(a, b, c, d, z) = \frac{az + b}{cz + d}$$

- `linFracTransExt` ($a, b, c, d \in \mathbb{C}$, $w \in \mathbb{C} \cup \{\infty\}$): The extended linear-fractional transformation on the Riemann sphere $\mathbb{C} \cup \{\infty\}$ (`OnePoint` $\mathbb{C}$).

```
noncomputable def linFracTransExt (a b c d : ℂ) : OnePoint ℂ → OnePoint ℂ
  | (z : ℂ) => if c * z + d = 0 then ∞ else (linFracTrans a b c d z : ℂ)
  | ∞ => if c = 0 then ∞ else (a / c : ℂ)
```

$$\varphi(w) = \begin{cases} \frac{az+b}{cz+d} & \text{if } w = z \in \mathbb{C} \text{ and } cz + d \neq 0, \\ \infty & \text{if } w = z \in \mathbb{C} \text{ and } cz + d = 0, \\ \infty & \text{if } w = \infty \text{ and } c = 0, \\ \frac{a}{c} & \text{if } w = \infty \text{ and } c \neq 0. \end{cases}$$

- `IsLinFracTrans` ($\varphi : \mathbb{C} \cup \{\infty\} \to \mathbb{C} \cup \{\infty\}$): Predicate stating that φ is a non-degenerate linear-fractional transformation on $\mathbb{C} \cup \{\infty\}$.

```
def IsLinFracTrans (φ : OnePoint ℂ → OnePoint ℂ) : Prop :=
  ∃ a b c d : ℂ, a * d - b * c ≠ 0 ∧ φ = linFracTransExt a b c d
```

$$\text{IsLinFracTrans}(\varphi) \iff \exists a, b, c, d \in \mathbb{C},\ ad - bc \neq 0 \ \wedge\ \varphi(w) = \frac{aw + b}{cw + d}$$

- `gl2OfCoeffs` ($a, b, c, d \in \mathbb{C}$, $ad - bc \neq 0$): The invertible 2×2 matrix in $\mathrm{GL}(2, \mathbb{C})$ associated with coefficients (a, b, c, d).

```
noncomputable def gl2OfCoeffs (a b c d : ℂ) (h_det : a * d - b * c ≠ 0) : GL
    (Fin 2) ℂ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero !![a, b; c, d] ...
```

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \mathrm{GL}(2, \mathbb{C}), \quad ad - bc \neq 0$$

- `translationMatrix` ($b \in \mathbb{C}$):

```
noncomputable def translationMatrix (b : ℂ) : GL (Fin 2) ℂ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero !![1, b; 0, 1] ...
```

$$\begin{pmatrix} 1 & b \\ 0 & 1 \end{pmatrix}$$

- `scalingMatrix` ($a \in \mathbb{C}, a \neq 0$):

```
noncomputable def scalingMatrix (a : ℂ) (ha : a ≠ 0) : GL (Fin 2) ℂ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero !![a, 0; 0, 1] ...
```

$$\begin{pmatrix} a & 0 \\ 0 & 1 \end{pmatrix}$$

- `inversionMatrix`:

```
noncomputable def inversionMatrix : GL (Fin 2) ℂ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero !![0, 1; 1, 0] ...
```

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

- `cross_ratio_Cinfty` ($z_1, z_2, z_3, z_4 \in \mathbb{C} \cup \{\infty\}$): The spherical cross-ratio $(z_1, z_2, z_3, z_4) \in \mathbb{C} \cup \{\infty\}$, defined as $\varphi(z_4)$ where φ is the unique linear-fractional transformation mapping $z_1 \mapsto \infty$, $z_2 \mapsto 0$, and $z_3 \mapsto 1$.

```
noncomputable def cross_ratio_Cinfty (z₁ z₂ z₃ z₄ : OnePoint ℂ) : OnePoint ℂ
    :=
  (Classical.epsilon (fun φ : OnePoint ℂ → OnePoint ℂ =>
    IsLinFracTrans φ ∧ φ z₁ = ∞ ∧ φ z₂ = (0 : ℂ) ∧ φ z₃ = (1 : ℂ))) z₄
```

$$(z_1, z_2, z_3, z_4) = \varphi(z_4) \quad \text{where } \varphi(z_1) = \infty,\ \varphi(z_2) = 0,\ \varphi(z_3) = 1$$

- `crossRatioFin` ($z_1, z_2, z_3, z_4 \in \mathbb{C}$): The rational cross-ratio formula for four finite complex numbers.

```
noncomputable def crossRatioFin (z₁ z₂ z₃ z₄ : ℂ) : ℂ :=
  ((z₄ - z₂) * (z₃ - z₁)) / ((z₄ - z₁) * (z₃ - z₂))
```

$$(z_1, z_2, z_3, z_4) = \frac{(z_4 - z_2)(z_3 - z_1)}{(z_4 - z_1)(z_3 - z_2)}$$

- `clineLocus` ($\alpha, \gamma \in \mathbb{R}$, $\beta \in \mathbb{C}$): The general locus of a circle or line (cline) in $\mathbb{C} \cup \{\infty\}$.

```
def clineLocus (α : ℝ) (β : ℂ) (γ : ℝ) : Set (OnePoint ℂ) :=
  {w : OnePoint ℂ | match w with
    | ∞ => α = 0
    | (z : ℂ) => α * ‖z‖ ^ 2 + 2 * (star β * z).re + γ = 0}
```

$$C_{\alpha,\beta,\gamma} = \{z \in \mathbb{C} : \alpha|z|^2 + 2\,\mathrm{Re}(\bar{\beta}z) + \gamma = 0\} \cup \{\infty \mid \alpha = 0\}$$

- `IsCline` ($S \subseteq \mathbb{C} \cup \{\infty\}$): Predicate stating that S is a non-degenerate cline ($|\beta|^2 > \alpha\gamma$).

```
def IsCline (S : Set (OnePoint ℂ)) : Prop :=
  ∃ (α : ℝ) (β : ℂ) (γ : ℝ), α * γ < ‖β‖ ^ 2 ∧ S = clineLocus α β γ
```

$$\mathrm{IsCline}(S) \iff \exists \alpha, \gamma \in \mathbb{R},\ \beta \in \mathbb{C},\ \alpha\gamma < |\beta|^2 \ \wedge\ S = C_{\alpha,\beta,\gamma}$$

- `circleReflect` ($z_0, z \in \mathbb{C}$, $R \in \mathbb{R}$): Reflection (inversion) across the circle $\{z \in \mathbb{C} : |z - z_0| = R\}$.

```
noncomputable def circleReflect (z₀ : ℂ) (R : ℝ) (z : ℂ) : ℂ :=
  z₀ + (R ^ 2 : ℂ) / star (z - z₀)
```

$$z^* = z_0 + \frac{R^2}{\overline{z - z_0}}$$

- `SymmetricWrtTriple` ($z_1, z_2, z_3, z, z^* \in \mathbb{C} \cup \{\infty\}$): Sarason's cross-ratio definition of symmetry with respect to the cline through (z_1, z_2, z_3).

```
def SymmetricWrtTriple (z₁ z₂ z₃ z z_star : OnePoint ℂ) : Prop :=
  cross_ratio_Cinfty z₁ z₂ z₃ z_star = conjExt (cross_ratio_Cinfty z₁ z₂ z₃ z)
```

$$(z_1, z_2, z_3, z^*) = \overline{(z_1, z_2, z_3, z)}$$

- `cayleyMap` ($z \in \mathbb{C}$): The Cayley transform mapping the upper half-plane $\mathbb{H} = \{\operatorname{Im} z > 0\}$ onto the unit disk $\mathbb{D} = \{|w| < 1\}$.

```
noncomputable def cayleyMap (z : ℂ) : ℂ :=
  linFracTrans 1 (-Complex.I) 1 Complex.I z
```

$$w = \frac{z - i}{z + i}$$

- `herglotzKernel` ($\zeta, z \in \mathbb{C}$): The Herglotz kernel mapping $z \in \mathbb{D}$ onto the right half-plane $\{\operatorname{Re} w > 0\}$ for $\zeta \in \partial\mathbb{D}$.

```
noncomputable def herglotzKernel (ζ z : ℂ) : ℂ :=
  (ζ + z) / (ζ - z)
```

$$H(\zeta, z) = \frac{\zeta + z}{\zeta - z}, \qquad \operatorname{Re} H(\zeta, z) = \frac{1 - |z|^2}{|\zeta - z|^2} \quad (|\zeta| = 1)$$

- `blaschkeFactor` ($a, z \in \mathbb{C}$): Disk automorphism / Blaschke factor mapping $\mathbb{D}$ onto $\mathbb{D}$ and taking $a \mapsto 0$.

```
noncomputable def blaschkeFactor (a z : ℂ) : ℂ :=
  (z - a) / (1 - star a * z)
```

$$\varphi_a(z) = \frac{z - a}{1 - \bar{a} z}$$

- `polarPoint` ($r, \theta \in \mathbb{R} \to \mathbb{C}$): Polar coordinate point $z = r\cos\theta + ir\sin\theta \in \mathbb{C}$, matching the $(r\cos\theta, r\sin\theta)$ parameterization of `polar_partial_r` and `polar_partial_theta`.

```
noncomputable def polarPoint (r θ : ℝ) : ℂ :=
  ((r * Real.cos θ : ℝ) : ℂ) + ((r * Real.sin θ : ℝ) : ℂ) * I
```

$$\text{polarPoint}(r, \theta) = r\cos\theta + ir\sin\theta = re^{i\theta}$$

- `IsStarlikeDomain_theta` ($\Omega \subseteq \mathbb{C}$): Geometric definition of a starlike domain with respect to the origin 0 in terms of the polar angle θ.

```
def IsStarlikeDomain_theta (Ω : Set ℂ) : Prop :=
  (0 : ℂ) ∈ Ω ∧
  ∀ (r θ : ℝ), 0 ≤ r → polarPoint r θ ∈ Ω →
    ∀ t : ℝ, 0 ≤ t → t ≤ 1 → polarPoint (t * r) θ ∈ Ω
```

IsStarlikeDomain_theta(Ω) $\iff 0 \in \Omega \wedge \forall r \geq 0,\ \forall \theta \in \mathbb{R},\ re^{i\theta} \in \Omega \implies \forall t \in [0,1],\ (tr)e^{i\theta} \in \Omega$

- `argPolarDeriv_theta` ($u, v, u_x, u_y, v_x, v_y : \mathbb{R}^2 \to \mathbb{R}$, $p = (r, \theta) \in \mathbb{R}^2 \to \mathbb{R}$): Angular rate of change $\frac{\partial \arg f(re^{i\theta})}{\partial \theta}$ of the image argument along the polar circle of radius r, defined directly via `polar_partial_theta`.

```
noncomputable def argPolarDeriv_theta (u v ux uy vx vy : ℝ × ℝ → ℝ) (p : ℝ ×
    ℝ) : ℝ :=
  let q : ℝ × ℝ := (p.1 * Real.cos p.2, p.1 * Real.sin p.2)
  (u q * polar_partial_theta vx vy p - v q * polar_partial_theta ux uy p) /
    (u q ^ 2 + v q ^ 2)
```

$$\frac{\partial \arg f(re^{i\theta})}{\partial \theta} = \frac{u(r\cos\theta, r\sin\theta)\, v_\theta(r,\theta) - v(r\cos\theta, r\sin\theta)\, u_\theta(r,\theta)}{u(r\cos\theta, r\sin\theta)^2 + v(r\cos\theta, r\sin\theta)^2} = \mathrm{Re}\left(\frac{zf'(z)}{f(z)}\right)$$

- `IsStarlikeOnDisk_theta_eps` ($f : \mathbb{C} \to \mathbb{C}$): Angle-based ($\theta$ / `polar_partial_theta`) and Weierstrass `_eps` definition of a starlike holomorphic map on the open unit disk $\mathbb{D} =$ `unitDiskSet`.

```
def IsStarlikeOnDisk_theta_eps (f : ℂ → ℂ) : Prop :=
  HolomorphicOn_eps f unitDiskSet ∧
  f 0 = 0 ∧
  (∀ z ∈ unitDiskSet, z ≠ 0 → f z ≠ 0) ∧
  ∀ (r θ : ℝ) (ux uy vx vy : ℝ × ℝ → ℝ),
    0 < r → r < 1 →
    HasPartialDerivX_C_to_R_eps (fun w => (f w).re) (ux (r * Real.cos θ, r *
    Real.sin θ)) (polarPoint r θ) →
    HasPartialDerivY_C_to_R_eps (fun w => (f w).re) (uy (r * Real.cos θ, r *
    Real.sin θ)) (polarPoint r θ) →
    HasPartialDerivX_C_to_R_eps (fun w => (f w).im) (vx (r * Real.cos θ, r *
    Real.sin θ)) (polarPoint r θ) →
    HasPartialDerivY_C_to_R_eps (fun w => (f w).im) (vy (r * Real.cos θ, r *
    Real.sin θ)) (polarPoint r θ) →
    0 < argPolarDeriv_theta
      (fun q => (f ((q.1 : ℂ) + (q.2 : ℂ) * I)).re)
      (fun q => (f ((q.1 : ℂ) + (q.2 : ℂ) * I)).im)
      ux uy vx vy (r, θ)
```

$$f \in \mathcal{O}_\varepsilon(\mathbb{D}), \quad f(0) = 0, \quad f(z) \neq 0 \text{ on } \mathbb{D}\setminus\{0\}, \quad \frac{\partial \arg f(re^{i\theta})}{\partial \theta} > 0 \quad \forall r \in (0,1),\ \theta \in \mathbb{R}$$

- `IsStarlikeLogDeriv_eps` ($f : \mathbb{C} \to \mathbb{C}$): Analytic logarithmic-derivative definition of a starlike holomorphic map on $\mathbb{D}$ using `HolomorphicOn_eps`, `DifferentiableAt_eps`, and `deriv_eps`.

```
def IsStarlikeLogDeriv_eps (f : ℂ → ℂ) : Prop :=
  HolomorphicOn_eps f unitDiskSet ∧
  f 0 = 0 ∧
  (∀ z ∈ unitDiskSet, z ≠ 0 → f z ≠ 0) ∧
  ∀ (z : ℂ), z ∈ unitDiskSet → z ≠ 0 →
    ∀ (hd : DifferentiableAt_eps f z), 0 < ((z * deriv_eps f z hd) / f z).re
```

$$f \in \mathcal{O}_\varepsilon(\mathbb{D}), \quad f(0) = 0, \quad \mathrm{Re}\left(\frac{zf'(z)}{f(z)}\right) > 0 \quad \forall z \in \mathbb{D}\setminus\{0\}$$

- `pommerenkeLogDeriv` ($\zeta : \mathrm{Fin}\, m \to \mathbb{C}$, $\sigma : \mathrm{Fin}\, m \to \mathbb{R}$, $z \in \mathbb{C} \to \mathbb{C}$): Herglotz–Pommerenke logarithmic derivative of a starlike radial slit map $f(z) = z\prod_{j=1}^{m}(1-\overline{\zeta_j}z)^{-\sigma_j}$.

```
noncomputable def pommerenkeLogDeriv {m : ℕ} (ζ : Fin m → ℂ) (σ : Fin m → ℝ)
    (z : ℂ) : ℂ :=
  ∑ j : Fin m, ((σ j / 2 : ℝ) : ℂ) * herglotzKernel (ζ j) z
```

$$\frac{zf'(z)}{f(z)} = \sum_{j=1}^{m} \frac{\sigma_j}{2}\frac{\zeta_j + z}{\zeta_j - z} = 1 + \sum_{j=1}^{m} \frac{\sigma_j \overline{\zeta_j} z}{1 - \overline{\zeta_j} z} \quad \left(\sum_{j=1}^{m} \sigma_j = 2\right)$$

- `pommerenkeFactorPhase`, `pommerenkeArcPhase`, & `pommerenkeSlitAngle` ($m, k \in \mathbb{N}$, $\varphi, \sigma : \mathbb{N} \to \mathbb{R}$, $t \in \mathbb{R} \to \mathbb{R}$): Half-angle boundary phase of $(1-e^{-i\varphi_j}e^{it})^{-\sigma_j}$ and the resulting constant slit angle θ_k on the k-th arc $(\varphi_{k-1}, \varphi_k)$, satisfying $\theta_{k+1}-\theta_k = \sigma_k\pi$ (`pommerenkeSlitAngle_succ_sub_eq_sigma`

```
noncomputable def pommerenkeFactorPhase (φ_j σ_j t : ℝ) (beforePole : Bool) :
    ℝ :=
  -σ_j * ((t - φ_j + (if beforePole then Real.pi else -Real.pi)) / 2)

noncomputable def pommerenkeArcPhase (m k : ℕ) (φ σ : ℕ → ℝ) (t : ℝ) : ℝ :=
  t + ∑ j ∈ Finset.range m, pommerenkeFactorPhase (φ j) (σ j) t (decide (k ≤
    j))

noncomputable def pommerenkeSlitAngle (m k : ℕ) (φ σ : ℕ → ℝ) : ℝ :=
  pommerenkeArcPhase m k φ σ 0
```

$$\theta_k = \sum_{j=0}^{m-1} \frac{\sigma_j\varphi_j}{2} + \sum_{j<k} \frac{\sigma_j\pi}{2} - \sum_{j\geq k} \frac{\sigma_j\pi}{2}, \qquad \Delta\theta_k = \theta_{k+1} - \theta_k = \sigma_k\pi$$

6 `SlitHarmonicMeasure.lean` (Section 8: Polar Angle θ & `_eps` Monotone Channel Statement)

- `radialSlit_theta` ($\rho, \theta \in \mathbb{R} \to \mathcal{P}(\mathbb{C})$): Radial slit $R_k = \{re^{i\theta} : r \geq \rho\}$ at polar angle θ starting at modulus ρ, parameterized via `polarPoint`.

```
def radialSlit_theta (ρ θ : ℝ) : Set ℂ :=
  {w : ℂ | ∃ r : ℝ, ρ ≤ r ∧ w = polarPoint r θ}
```

$$R(\rho, \theta) = \{re^{i\theta} \in \mathbb{C} : r \geq \rho\}$$

- `angularChannel_theta` ($\theta_{\text{low}}, \theta_{\text{high}} \in \mathbb{R} \to \mathcal{P}(\mathbb{C})$): Open angular channel sector C_k between adjacent radial slit angles θ_{low} and θ_{high}.

```
def angularChannel_theta (θ_low θ_high : ℝ) : Set ℂ :=
  {w : ℂ | ∃ (r θ : ℝ), 0 < r ∧ θ_low < θ ∧ θ < θ_high ∧ w = polarPoint r θ}
```

$$C(\theta_{\text{low}}, \theta_{\text{high}}) = \{re^{i\theta} \in \mathbb{C} : r > 0,\ \theta_{\text{low}} < \theta < \theta_{\text{high}}\}$$

- `slitComplementDomain_theta` ($\rho, \theta : \mathrm{Fin}\, m \to \mathbb{R} \to \mathcal{P}(\mathbb{C})$): Radially slitted plane $\Omega = \mathbb{C} \setminus \bigcup_{j=1}^{m} R(\rho_j, \theta_j)$.

```
def slitComplementDomain_theta {m : ℕ} (ρ θ : Fin m → ℝ) : Set ℂ :=
  (⋃ j : Fin m, radialSlit_theta (ρ j) (θ j))ᶜ
```

$$\Omega = \mathbb{C} \setminus \bigcup_{j=1}^{m} \{re^{i\theta_j} : r \geq \rho_j\}$$

- `IsRealHarmonicOn_eps` ($\omega : \mathbb{C} \to \mathbb{R}$, $G \subseteq \mathbb{C}$): Harmonicity of a real-valued function on an open subset $G \subseteq \mathbb{C}$ via `Harmonic_R2_eps`.

```
def IsRealHarmonicOn_eps (ω : ℂ → ℝ) (G : Set ℂ) : Prop :=
  Harmonic_R2_eps (fun p : ℝ × ℝ => ω ((p.1 : ℂ) + (p.2 : ℂ) * I))
    {p : ℝ × ℝ | (p.1 : ℂ) + (p.2 : ℂ) * I ∈ G}
```

$$\Delta\omega = \omega_{xx} + \omega_{yy} = 0 \quad \text{on } G$$

- `poissonKernel_theta` ($\varphi \in \mathbb{R}$, $z \in \mathbb{C} \to \mathbb{R}$): Poisson kernel on $\mathbb{D}$ with boundary pole at polar angle φ ($\zeta = e^{i\varphi} =$ `polarPoint 1` φ), defined as the real part of `herglotzKernel`.

```
noncomputable def poissonKernel_theta (φ : ℝ) (z : ℂ) : ℝ :=
  (herglotzKernel (polarPoint 1 φ) z).re
```

$$P(z, e^{i\varphi}) = \mathrm{Re}\left(\frac{e^{i\varphi} + z}{e^{i\varphi} - z}\right) = \frac{1 - |z|^2}{|e^{i\varphi} - z|^2}$$

- `diskArcHarmonicMeasureOrigin` ($\varphi_1, \varphi_2 \in \mathbb{R} \to \mathbb{R}$): Normalized harmonic measure $\omega(0; (e^{i\varphi_1}, e^{i\varphi_2}); \mathbb{D})$ at the origin of a boundary arc on $\partial\mathbb{D}$.

```
noncomputable def diskArcHarmonicMeasureOrigin (φ₁ φ₂ : ℝ) : ℝ :=
  (φ₂ - φ₁) / (2 * Real.pi)
```

$$\omega(0; (e^{i\varphi_1}, e^{i\varphi_2}); \mathbb{D}) = \frac{\varphi_2 - \varphi_1}{2\pi}$$

- `AnalyticOrderedSlitConfiguration_eps` ($m \in \mathbb{N}$): Structure bundling a starlike conformal map $f : \mathbb{C} \to \mathbb{C}$ satisfying `IsStarlikeOnDisk_theta_eps`, the Herglotz–Pommerenke `deriv_eps` logarithmic derivative identity, critical tip preimages `HasDerivAt_R_to_C_eps`, interlaced boundary angles $\varphi_k \le t_k \le \varphi_{k+1}$, channel opening angles $\theta_{k+1} - \theta_k = \sigma_k \pi$, and unit-circle slit tips $\rho_k = 1$.

```
structure AnalyticOrderedSlitConfiguration_eps (m : ℕ) where
  f : ℂ → ℂ
  phi : ℕ → ℝ
  tipArg : ℕ → ℝ
  thetaSlit : ℕ → ℝ
  sigma : ℕ → ℝ
  rho : ℕ → ℝ
  hStarlike_theta : IsStarlikeOnDisk_theta_eps f
  hLogDeriv_eps :
    ∀ (z : ℂ), z ∈ unitDiskSet → z ≠ 0 → ∀ (hd : DifferentiableAt_eps f z),
      (z * deriv_eps f z hd) / f z =
        pommerenkeLogDeriv (fun j : Fin m => polarPoint 1 (phi j.val)) (fun j
    : Fin m => sigma j.val) z
  hTipCritical_eps : ∀ k : ℕ, HasDerivAt_R_to_C_eps (fun θ => f (polarPoint 1
    θ)) 0 (tipArg k)
  hInterlaceLeft : ∀ k : ℕ, phi k ≤ tipArg k
  hInterlaceRight : ∀ k : ℕ, tipArg k ≤ phi (k + 1)
  hChannelOpeningAngle : ∀ k : ℕ, thetaSlit (k + 1) - thetaSlit k = sigma k *
     Real.pi
  hUnitCircleTips : ∀ k : ℕ, rho k = 1
```

$$\frac{zf'(z)}{f(z)} = \sum_{j=1}^{m} \frac{\sigma_j}{2} \frac{e^{i\varphi_j} + z}{e^{i\varphi_j} - z}, \qquad \frac{d}{d\theta} f(e^{i\theta})\Big|_{\theta = t_k} = 0, \qquad \varphi_k \le t_k \le \varphi_{k+1}, \qquad \Delta\theta_k = \theta_{k+1} - \theta_k = \sigma_k \pi$$

- `MonotoneChannelStatement_eps` (C : `AnalyticOrderedSlitConfiguration_eps` m, $N \in \mathbb{N}$): The Analytic Monotone Channel Harmonic Measure Statement (Section 2, Equation 2.21 / Lemma 2.1).

```
def MonotoneChannelStatement_eps {m : ℕ} (C :
    AnalyticOrderedSlitConfiguration_eps m) (N : ℕ) : Prop :=
  (∀ k < N, analyticChannelOpeningAngle_theta C k ≤
    analyticChannelOpeningAngle_theta C (k + 1)) →
  (∀ k < N, analyticChannelHarmonicMeasure_eps C k ≤
    analyticChannelHarmonicMeasure_eps C (k + 1))
```

$$(\forall k < N,\ \Delta\theta_k \le \Delta\theta_{k+1}) \implies \left(\forall k < N,\ \omega(0;\partial C_k;\Omega) = \frac{t_{k+1}-t_k}{2\pi} \le \frac{t_{k+2}-t_{k+1}}{2\pi} = \omega(0;\partial C_{k+1};\Omega)\right)$$

- `MonotoneSlitStatement_eps` (C : `AnalyticOrderedSlitConfiguration_eps` m, $N \in \mathbb{N}$): The Analytic Monotone Slit Harmonic Measure Statement (Section 1, Question 1.1 / Lemma 2.2).

```
def MonotoneSlitStatement_eps {m : ℕ} (C :
    AnalyticOrderedSlitConfiguration_eps m) (N : ℕ) : Prop :=
  (∀ k < N, analyticChannelOpeningAngle_theta C k ≤
    analyticChannelOpeningAngle_theta C (k + 1)) →
  (∀ k < N, analyticSlitHarmonicMeasure_eps C k ≤
    analyticSlitHarmonicMeasure_eps C (k + 1))
```

$$(\forall k < N,\ \Delta\theta_k \le \Delta\theta_{k+1}) \implies \left(\forall k < N,\ \omega(0;R_k;\Omega) = \frac{\varphi_{k+1}-\varphi_k}{2\pi} \le \frac{\varphi_{k+2}-\varphi_{k+1}}{2\pi} = \omega(0;R_{k+1};\Omega)\right)$$

- `riemannSlitMap_f` ($n \in \mathbb{N}$, $z_{\mathrm{pt}} : \mathbb{N} \to \mathbb{C}$, $\sigma : \mathbb{N} \to \mathbb{R}$, $z \in \mathbb{C} \to \mathbb{C}$): Explicit Pommerenke formula (Section 1, Equation 1.1 of *Discretization of a Harmonic Measure Problem and Spandrel*) for the Riemann map $f : \mathbb{D} \to \Omega$.

```
noncomputable def riemannSlitMap_f (n : ℕ) (z_pt : ℕ → ℂ) (σ : ℕ → ℝ) (z :
    ℂ) : ℂ :=
  z * ∏ j ∈ Finset.Icc 1 (2 * n), (1 - star (z_pt j) * z) ^ (-(σ j : ℂ))
```

$$f(z) = z\prod_{j=1}^{2n}(1-\bar{z}_j z)^{-\sigma_j}$$

- `PaperSection1SymmetricSlitSetup_eps` ($n \in \mathbb{N}$): Exact symmetrical setup of Section 1 (Equations 1.1–1.6 of `IsoRadStarQuestion.tex`), including the real-axis conjugation symmetries for $k = 2, 3, \ldots, n$ and equal starting moduli $\rho_k = \rho_j$.

```
structure PaperSection1SymmetricSlitSetup_eps (n : ℕ) where
  f : ℂ → ℂ
  φ : ℕ → ℝ
  z_pt : ℕ → ℂ
  σ : ℕ → ℝ
  t : ℕ → ℝ
  ρ : ℕ → ℝ
  hStarlike_theta : IsStarlikeOnDisk_theta_eps f
  hUnitCircle_z : ∀ k : ℕ, z_pt k = polarPoint 1 (φ k)
  hFormula_f : ∀ z ∈ unitDiskSet, f z = riemannSlitMap_f n z_pt σ z
  hSymm_z : ∀ k : ℕ, 2 ≤ k → k ≤ n → z_pt (2 * n + 2 - k) = star (z_pt k)
  hSymm_sigma : ∀ k : ℕ, 2 ≤ k → k ≤ n → σ (2 * n + 2 - k) = σ k
  hSymm_tip : ∀ k : ℕ, 2 ≤ k → k ≤ n → polarPoint 1 (t (2 * n + 1 - k)) =
    polarPoint 1 (-t k)
  hSymm_rho : ∀ k : ℕ, 2 ≤ k → k ≤ n → ρ (2 * n + 1 - k) = ρ k
  hEqualRho : ∀ j k : ℕ, ρ k = ρ j
```

$$z_{2n+2-k} = \bar{z}_k, \qquad \sigma_{2n+2-k} = \sigma_k, \qquad \exp(it_{2n+1-k}) = \exp(-it_k), \qquad \rho_{2n+1-k} = \rho_k, \qquad \rho_k = \rho_j$$

- `paperSlitArcLength_omega` (S : `PaperSection1SymmetricSlitSetup_eps` n, $k \in \mathbb{N} \to \mathbb{R}$): Arc lengths $\omega_k = |(z_k, z_{k+1})|$ on the unit circle from z_k to z_{k+1} (Section 1 of `IsoRadStarQuestion.tex`).

```
def paperSlitArcLength_omega {n : ℕ} (S : PaperSection1SymmetricSlitSetup_eps
    n) (k : ℕ) : ℝ :=
  S.φ (k + 1) - S.φ k
```

$$\omega_k = |(z_k, z_{k+1})| = \varphi_{k+1} - \varphi_k$$

- `Question_1_1_eps` ($n \in \mathbb{N}$, S : `PaperSection1SymmetricSlitSetup_eps` n): Section 1, Question 1.1 of `IsoRadStarQuestion.tex` (*"Does $\{\omega_k\}_1^n$ being monotone imply $\{\sigma_k\}_1^n$ are monotone, and vice versa?"*).

```
def Question_1_1_eps (n : ℕ) (S : PaperSection1SymmetricSlitSetup_eps n) :
    Prop :=
  (∀ k : ℕ, 1 ≤ k → k < n → paperSlitArcLength_omega S k ≤
   paperSlitArcLength_omega S (k + 1)) ↔
  (∀ k : ℕ, 1 ≤ k → k < n → S.σ k ≤ S.σ (k + 1))
```

$$(\forall k \in \{1, \dots, n-1\},\ \omega_k \le \omega_{k+1}) \iff (\forall k \in \{1, \dots, n-1\},\ \sigma_k \le \sigma_{k+1})$$

- `symmetricSlit_S_k` ($\theta : \mathbb{N} \to \mathbb{R}$, $k \in \mathbb{N} \to \mathcal{P}(\mathbb{C})$): Symmetric slits starting on the unit circle (Section 2, Equation 2.1 of `IsoRadStarQuestion.tex`).

```
def symmetricSlit_S_k (θ : ℕ → ℝ) (k : ℕ) : Set ℂ :=
  radialSlit_theta 1 (θ k) ∪ radialSlit_theta 1 (-θ k)
```

$$S_k = \{re^{\pm i\theta_k} : r \ge 1\}$$

- `MonotoneSymmetricSlitAngles` ($N \in \mathbb{N}$, $\theta : \mathbb{N} \to \mathbb{R}$): Monotone slit angle hypothesis $\theta_0 = 0 < \theta_1 < \theta_2 < \cdots < \theta_N$ with $(\theta_k - \theta_{k-1}) \le (\theta_{k+1} - \theta_k)$ for $k = 1, \dots, N-1$ (Section 2 of `IsoRadStarQuestion.tex`).

```
def MonotoneSymmetricSlitAngles (N : ℕ) (θ : ℕ → ℝ) : Prop :=
  θ 0 = 0 ∧
  (∀ k < N, θ k < θ (k + 1)) ∧
  (∀ k : ℕ, 1 ≤ k → k < N → θ k - θ (k - 1) ≤ θ (k + 1) - θ k)
```

$$\theta_0 = 0 < \theta_1 < \theta_2 < \cdots < \theta_N, \qquad (\theta_k - \theta_{k-1}) \le (\theta_{k+1} - \theta_k) \quad (k = 1, \dots, N-1)$$

- `slitDomain_Omega` ($N \in \mathbb{N}$, $\theta : \mathbb{N} \to \mathbb{R} \to \mathcal{P}(\mathbb{C})$): Symmetric slitted starlike domain $\Omega = \mathbb{C} \setminus \left(\bigcup_{k=0}^{N} S_k\right)$ (Section 2, Equation 2.2 of `IsoRadStarQuestion.tex`).

```
def slitDomain_Omega (N : ℕ) (θ : ℕ → ℝ) : Set ℂ :=
  (⋃ k ∈ Finset.range (N + 1), symmetricSlit_S_k θ k)ᶜ
```

$$\Omega = \mathbb{C} \setminus \left(\bigcup_{k=0}^{N} S_k\right)$$

- `channel_C_k` ($\theta_{\text{seq}} : \mathbb{N} \to \mathbb{R}$, $k \in \mathbb{N} \to \mathcal{P}(\mathbb{C})$): Symmetric channels outside the unit disk (Section 2, Equation 2.3 of `IsoRadStarQuestion.tex`).

```
def channel_C_k (θ_seq : ℕ → ℝ) (k : ℕ) : Set ℂ :=
  {w : ℂ | ∃ (r θ : ℝ), 1 < r ∧ θ_seq k < θ ∧ θ < θ_seq (k + 1) ∧
    (w = polarPoint r θ ∨ w = polarPoint r (-θ))}
```

$$C_k = \{re^{\pm i\theta} : r > 1,\ \theta_k < \theta < \theta_{k+1}\}$$

- `isoradialModulusStep` ($m \in \mathbb{N}$, $\delta, r_j \in \mathbb{R} \to \mathbb{R}$): Recursive modulus formula for the polar isoradial graph Γ^δ (Section 2, Equation 2.4 of `IsoRadStarQuestion.tex`).

```
noncomputable def isoradialModulusStep (m : ℕ) (δ r_j : ℝ) : ℝ :=
  r_j * Real.cos (Real.pi / (m : ℝ)) +
    Real.sqrt ((r_j * Real.cos (Real.pi / (m : ℝ))) ^ 2 - r_j ^ 2 + δ ^ 2)
```

$$r_{j+1} = r_j \cos(\pi/m) + \sqrt{(r_j \cos(\pi/m))^2 - r_j^2 + \delta^2}$$

- `isoradialGraphVertices_V_Gamma_delta` ($m \in \mathbb{N}$, $\delta \in \mathbb{R}$, $r_{\text{seq}} : \mathbb{N} \to \mathbb{R} \to \mathcal{P}(\mathbb{C})$): Vertex set $V(\Gamma^\delta)$ of the polar isoradial graph (Section 2, Equation 2.5 of `IsoRadStarQuestion.tex`).

```
def isoradialGraphVertices_V_Gamma_delta (m : ℕ) (δ : ℝ) (r_seq : ℕ → ℝ) :
    Set ℂ :=
  {v : ℂ | ∃ (n ℓ : ℕ), 1 ≤ ℓ ∧ ℓ ≤ m ∧ v = polarPoint (r_seq (2 * n)) ((ℓ : ℝ
    ) * δ ^ 2)}
```

$$V(\Gamma^\delta) = \{r_j e^{i\ell\delta^2} : \ell = 1, \ldots, m, \qquad r_j \in \{r_{2n}\}\}$$

- `discreteSlit_S_k_delta` ($V_\Gamma \subseteq \mathbb{C}$, $\gamma^\delta : \mathbb{N} \to \mathbb{C}$, $k \in \mathbb{N} \to \mathcal{P}(\mathbb{C})$): Discrete slit $(S_k)_{\Omega^\delta}$ defined via arg from Chapter 2 (Section 2, Equation 2.8 of `IsoRadStarQuestion.tex`).

```
def discreteSlit_S_k_delta (V_Gamma : Set ℂ) (γ_delta : ℕ → ℂ) (k : ℕ) : Set
    ℂ :=
  {v ∈ V_Gamma | arg v = arg (γ_delta k) ∧ ‖ γ_delta k ‖ ≤ ‖ v ‖}
```

$$(S_k)_{\Omega^\delta} = \{v \in V(\Gamma^\delta) : \arg v = \arg \gamma_k^\delta,\ |v| \geq |\gamma_k^\delta|\}$$

- `discreteChannel_C_k_delta` ($V_\Gamma \subseteq \mathbb{C}$, $n_k : \mathbb{N} \to \mathbb{N}$, $v^\delta : \mathbb{N} \to \mathbb{N} \to \mathbb{C}$, $r_{\max}^\delta \in \mathbb{R}$, $k \in \mathbb{N} \to \mathcal{P}(\mathbb{C})$): Discrete channel $(C_k)_{\Omega^\delta}$ (Section 2, Equation 2.13 of `IsoRadStarQuestion.tex`).

```
def discreteChannel_C_k_delta (V_Gamma : Set ℂ) (n_k : ℕ → ℕ) (v_open : ℕ →
    ℕ → ℂ)
    (r_max : ℝ) (k : ℕ) : Set ℂ :=
  ⋃ ℓ ∈ Finset.Icc 1 (n_k k),
    {v ∈ V_Gamma | arg v = arg (v_open k ℓ) ∧ ‖ v_open k ℓ ‖ ≤ ‖ v ‖ ∧ ‖ v ‖ <
    r_max}
```

$$(C_k)_{\Omega^\delta} = \bigcup_{\ell=1}^{n_k} \{v \in V(\Gamma^\delta) : \arg v = \arg v_{k,\ell}^\delta, \quad |v_{k,\ell}^\delta| \leq |v| < r_{\max}^\delta\}$$

- `discreteChannelBoundary_delta` ($\partial\Omega_\Gamma^\delta \subseteq \mathbb{C}$, $E_\Gamma : \mathbb{C} \to \mathbb{C} \to \text{Prop}$, $(C_k)_{\Omega^\delta} \subseteq \mathbb{C} \to \mathcal{P}(\mathbb{C})$): Discrete channel boundary $\partial(C_k)_{\Omega^\delta}$ (Section 2, Equation 2.20 of `IsoRadStarQuestion.tex`).

```
def discreteChannelBoundary_delta (bdry_Omega_delta : Set ℂ) (E_Gamma : ℂ →
    ℂ → Prop)
    (C_k_delta : Set ℂ) : Set ℂ :=
  {v ∈ bdry_Omega_delta | ∃ u ∈ C_k_delta, E_Gamma v u}
```

$$\partial(C_k)_{\Omega^\delta} = \{v \in \partial\Omega_\Gamma^\delta : (v, u) \in E(\Gamma^\delta), \quad u \in (C_k)_{\Omega^\delta}\}$$

- `markovFourTermStep` ($m_l, m_r, m_u, m_d, p_l, p_r, p_u, p_d \in \mathbb{N} \to \mathbb{N}$): Four-neighbor Markov process equation for $RW(t+1)$ on Ω_Γ^δ (Section 2, Equations 2.26–2.27 of `IsoRadStarQuestion.tex`).

```
def markovFourTermStep (ml mr mu md pl pr pu pd : Nat) : Nat :=
  ml * pl + mr * pr + mu * pu + md * pd
```

$\mathbf{P}(RW(t+1) = v) = M_{v,v_l}\mathbf{P}(RW(t) = v_l) + M_{v,v_r}\mathbf{P}(RW(t) = v_r) + M_{v,v_u}\mathbf{P}(RW(t) = v_u) + M_{v,v_d}\mathbf{P}(RW(t) = v$

- `IsInfiniteDyadicPolarIsoradialStarlikeGraph` & `Question_2_3_Spandrel` ($m \in \mathbb{N}$, $\delta \in \mathbb{R}$, $V_\Gamma \subseteq \mathbb{C}$): Definition of an *infinite dyadic polar isoradial starlike graph* and Question 2.3 (Spandrel) from Section 2 of `IsoRadStarQuestion_Lean2026.tex`, expressed via `arg` and `polarPoint`.

```
def IsInfiniteDyadicPolarIsoradialStarlikeGraph (m : ℕ) (δ : ℝ) (V_Gamma :
    Set ℂ) : Prop :=
  (∀ v ∈ V_Gamma, ∃ n : ℤ, arg v = (2 * Real.pi) / (2 : ℝ) ^ n) ∧
  (∀ r θ : ℝ, polarPoint r θ ∈ V_Gamma →
    polarPoint (isoradialModulusStep m δ r) θ ∈ V_Gamma) ∧
  (∀ r : ℝ, 0 < r → ∃ v ∈ V_Gamma, r < ‖ v ‖)

def Question_2_3_Spandrel (m : ℕ) (δ : ℝ) : Prop :=
  ∃ V_Gamma : Set ℂ, IsInfiniteDyadicPolarIsoradialStarlikeGraph m δ V_Gamma
```

(Dyadic) $v \in V(\Gamma^\delta) \implies \arg v = 2\pi/2^n$ for some $n \in \mathbb{Z}$,

(Starlike) $re^{i\theta} \in V(\Gamma^\delta) \implies r'e^{i\theta} \in V(\Gamma^\delta), \quad r' = r\cos(\pi/m) + \sqrt{(r\cos(\pi/m))^2 - r^2 + \delta^2}$,

(Infinite) $\forall r > 0,\ \exists v \in V(\Gamma^\delta)$ such that $|v| > r$.

- `isoradialTangentWeight` ($\theta_k, \theta_{\text{sum}} \in \mathbb{R} \to \mathbb{R}$): Chelkak–Smirnov isoradial random walk transition weight (Section 2, Equation 2.19 of `IsoRadStarQuestion.tex`).

```
noncomputable def isoradialTangentWeight (θ_k θ_sum : ℝ) : ℝ :=
  Real.tan θ_k / θ_sum
```

$$\mathbf{P}(\xi_v = u_k - v) = \frac{\tan\theta_k}{\sum_j \tan\theta_j}$$

- `markovFourTermStep_R` ($m_l, m_r, m_u, m_d, p_l, p_r, p_u, p_d \in \mathbb{R} \to \mathbb{R}$): Real-valued four-neighbor Markov transition operator on Ω^δ_Γ (Section 2, Equations 2.26–2.27 of `IsoRadStarQuestion.tex`).

```
def markovFourTermStep_R (ml mr mu md pl pr pu pd : ℝ) : ℝ :=
  ml * pl + mr * pr + mu * pu + md * pd
```

$\mathbf{P}(RW(t{+}1) = v) = M_{v,v_l}\mathbf{P}(RW(t) = v_l) + M_{v,v_r}\mathbf{P}(RW(t) = v_r) + M_{v,v_u}\mathbf{P}(RW(t) = v_u) + M_{v,v_d}\mathbf{P}(RW(t) = v$

- `discreteChannelWidth_delta` ($\Delta\theta, \delta \in \mathbb{R} \to \mathbb{N}$): Discrete channel width $n_k^\delta = \lfloor(\theta_{k+1} - \theta_k)/\delta^2\rfloor$ (Section 2, Equation 2.12 of `IsoRadStarQuestion.tex`).

```
noncomputable def discreteChannelWidth_delta (Δθ δ : ℝ) : ℕ :=
  Nat.floor (Δθ / δ ^ 2)
```

$$n_k^\delta = \left\lfloor \frac{\theta_{k+1} - \theta_k}{\delta^2} \right\rfloor$$

- `discreteChannelHarmonicMeasure_R` ($n_k^\delta \in \mathbb{N}$, $p_{\text{ray}} : \mathbb{N} \to \mathbb{R} \to \mathbb{R}$): Real-valued discrete channel harmonic measure $\omega^\delta(0; \partial(C_k)_{\Omega^\delta}; \Omega^\delta_\Gamma)$ (Section 2, Equation 2.21 of `IsoRadStarQuestion.tex`).

```
def discreteChannelHarmonicMeasure_R (n_width : ℕ) (p_ray : ℕ → ℝ) : ℝ :=
  ∑ ℓ ∈ Finset.Icc 1 n_width, p_ray ℓ
```

$$\omega^\delta(0; \partial(C_k)_{\Omega^\delta}; \Omega^\delta_\Gamma) = \sum_{\ell=1}^{n_k^\delta} p^\delta_{k,\ell}$$

- `analyticSlitTipCotangentSum` (C : `AnalyticOrderedSlitConfiguration_eps` m, $t \in \mathbb{R} \to \mathbb{R}$): Boundary cotangent sum characterizing the critical slit-tip preimages $e^{it_k} \in \partial\mathbb{D}$.

```
noncomputable def analyticSlitTipCotangentSum {m : ℕ}
    (C : AnalyticOrderedSlitConfiguration_eps m) (t : ℝ) : ℝ :=
  ∑ j : Fin m, (C.sigma j.val / 2) * (Real.sin (t - C.phi j.val) / (1 - Real.
    cos (t - C.phi j.val)))
```

$$\operatorname{Im}\left(\frac{e^{it}f'(e^{it})}{f(e^{it})}\right)=\sum_{j=1}^{m}\frac{\sigma_j}{2}\frac{\sin(t-\varphi_j)}{1-\cos(t-\varphi_j)}=\sum_{j=1}^{m}\frac{\sigma_j}{2}\cot\left(\frac{t-\varphi_j}{2}\right)$$

- `IsoradialContinuumBridge_eps` (C : `AnalyticOrderedSlitConfiguration_eps` m): Structure packaging the Lemma 2.1/2.2 ray dominance with the Chelkak–Smirnov (`HasLimitAt_eps` as $\delta \to 0$) and Beurling–Nevanlinna (`HasLimitAtInf_eps` as $\|w\| \to \infty$) convergence limits used by `monotoneChannelStatement_eps_proven`.

```
structure IsoradialContinuumBridge_eps {m : ℕ} (C :
    AnalyticOrderedSlitConfiguration_eps m) where
  R₀ : ℝ
  δ₀ : ℝ
  hδ₀_pos : 0 < δ₀
  p_ray : ℂ → ℂ → ℕ → ℕ → ℝ
  ω_trunc : ℕ → ℂ → ℝ
  hRayNonneg : ∀ (w δ : ℂ) (k ℓ : ℕ), 0 ≤ p_ray w δ k ℓ
  hRayDom : ∀ (w δ : ℂ) (k : ℕ),
    analyticChannelOpeningAngle_theta C k ≤ analyticChannelOpeningAngle_theta
     C (k + 1) →
    ∀ ℓ ∈ Finset.Icc 1 (discreteChannelWidth_delta (
    analyticChannelOpeningAngle_theta C k) ‖ δ ‖),
      p_ray w δ k ℓ ≤ p_ray w δ (k + 1) ℓ
  hChelkakSmirnov_eps : ∀ (w : ℂ) (k : ℕ), ‖ w ‖ > R₀ →
    HasLimitAt_eps
      (fun δ : ℂ =>
        ((discreteChannelHarmonicMeasure_R
          (discreteChannelWidth_delta (analyticChannelOpeningAngle_theta C k)
   ‖ δ ‖)
          (p_ray w δ k) : ℝ) : ℂ))
      (ω_trunc k w : ℂ) 0
  hBeurlingNevanlinna_eps : ∀ k : ℕ,
    HasLimitAtInf_eps (ω_trunc k) (analyticChannelHarmonicMeasure_eps C k)
```

$$\lim_{\|w\|\to\infty}\lim_{\delta\to 0}\sum_{\ell=1}^{n_k^\delta}p_{k,\ell}^{\delta}(w)=\omega(0;\partial C_k;\Omega)$$

- `IsoradialVertexStar` ($d \in \mathbb{N}$): Chelkak–Smirnov isoradial vertex star at $v \in \mathbb{C}$ with d neighbors $u_k \in \mathbb{C}$, rhombic half-angles $\theta_k \in \mathbb{R}$, and dual circumcenters $w_{\mathrm{next}}(k), w_{\mathrm{prev}}(k) \in \mathbb{C}$ satisfying $(\tan\theta_k)(u_k - v) = -i(w_{\mathrm{next}}(k) - w_{\mathrm{prev}}(k))$ and $\sum_{k=1}^{d}(w_{\mathrm{next}}(k) - w_{\mathrm{prev}}(k)) = 0$ (Dmitry Chelkak, `https://www-personal.umich.edu/~chelkak/`, and Stanislav Smirnov, `http://www.unige.ch/~smirnov/`, *Discrete complex analysis on isoradial graphs*, Adv. Math. **228** (2011), 1590–1630, `arXiv:0810.2188`, Sec. 2.1).

```
structure IsoradialVertexStar (d : ℕ) where
  v : ℂ
  u : Fin d → ℂ
  theta : Fin d → ℝ
  w_next : Fin d → ℂ
  w_prev : Fin d → ℂ
  hTanPos : ∀ k : Fin d, 0 < Real.tan (theta k)
  hTanSumPos : 0 < ∑ k : Fin d, Real.tan (theta k)
  hAreaPos : 0 < (1 / 4 : ℝ) * ∑ k : Fin d,
    Real.tan (theta k) * ((u k - v).re ^ 2 + (u k - v).im ^ 2)
  hDualOrthog : ∀ k : Fin d,
    ((Real.tan (theta k) : ℝ) : ℂ) * (u k - v) = -I * (w_next k - w_prev k)
  hDualCycle : ∑ k : Fin d, (w_next k - w_prev k) = 0
```

$$(\tan\theta_k)(u_k - v) = -i(w_k - w_{k-1}), \qquad \sum_{k=1}^{d} \tan\theta_k\,(u_k - v) = -i\sum_{k=1}^{d}(w_k - w_{k-1}) = 0$$

- `isoradialDiscreteMeanValue` (S : `IsoradialVertexStar` d, $H : \mathbb{C} \to \mathbb{R} \to \mathbb{R}$): Chelkak–Smirnov discrete Mean Value operator $(M^\delta H)(v) = \sum_{k=1}^{d} P(\xi_v = u_k - v)\, H(u_k)$.

```
noncomputable def isoradialDiscreteMeanValue {d : ℕ}
    (S : IsoradialVertexStar d) (H : ℂ → ℝ) : ℝ :=
  ∑ k : Fin d, isoradialStarProb S k * H (S.u k)
```

$$(M^\delta H)(v) = \sum_{k=1}^{d} \left(\frac{\tan\theta_k}{\sum_{j=1}^{d} \tan\theta_j} \right) H(u_k)$$

- `isoradialDiscreteLaplacian` & `IsDiscreteHarmonicAt` (S : `IsoradialVertexStar` d, $H : \mathbb{C} \to \mathbb{R}$): Chelkak–Smirnov discrete Laplacian $(\Delta^\delta H)(v)$ and discrete harmonicity predicate $(\Delta^\delta H)(v) = 0 \iff H(v) = (M^\delta H)(v)$.

```
noncomputable def isoradialDiscreteLaplacian {d : ℕ}
    (S : IsoradialVertexStar d) (H : ℂ → ℝ) : ℝ :=
  (1 / isoradialStarAreaWeight S) *
    ∑ k : Fin d, Real.tan (S.theta k) * (H (S.u k) - H S.v)

def IsDiscreteHarmonicAt {d : ℕ} (S : IsoradialVertexStar d) (H : ℂ → ℝ) :
    Prop :=
  isoradialDiscreteLaplacian S H = 0
```

$$(\Delta^\delta H)(v) = \frac{1}{\mu_\Gamma(v)} \sum_{k=1}^{d} \tan\theta_k \big(H(u_k) - H(v)\big), \qquad (\Delta^\delta H)(v) = 0 \iff H(v) = (M^\delta H)(v)$$

- `secondOrderTaylorAt` & `IsQuadraticallyBalancedStar` ($v \in \mathbb{C}$, $h_0, h_x, h_y, h_{xx}, h_{xy}, h_{yy} \in \mathbb{R}$, S : `IsoradialVertexStar` d): Second-order Taylor polynomial $T_2(z)$ centered at v and quadratic balance condition $\sum_{k=1}^{d} \tan\theta_k\,(u_k - v)^2 = 0$ ensuring $(\Delta^\delta T_2)(v) = h_{xx} + h_{yy}$.

```
noncomputable def secondOrderTaylorAt (v : ℂ) (h₀ hx hy hxx hxy hyy : ℝ) (z :
    ℂ) : ℝ :=
  let dx := (z - v).re
  let dy := (z - v).im
  h₀ + hx * dx + hy * dy + (1 / 2 : ℝ) * hxx * dx ^ 2 + hxy * dx * dy + (1 / 2
    : ℝ) * hyy * dy ^ 2

def IsQuadraticallyBalancedStar {d : ℕ} (S : IsoradialVertexStar d) : Prop :=
  (∑ k : Fin d, Real.tan (S.theta k) * ((S.u k - S.v).re ^ 2 - (S.u k - S.v).
    im ^ 2) = 0) ∧
  (∑ k : Fin d, Real.tan (S.theta k) * ((S.u k - S.v).re * (S.u k - S.v).im)
    = 0)
```

$$(\Delta^\delta T_2)(v) = h_{xx} + h_{yy}, \qquad (\Delta^\delta h)(v) = (h_{xx} + h_{yy}) + \Delta^\delta(h - T_2)(v)$$